Amazon SDE Last-Minute DSA Add-On

LC 33 + LC 236 | Interview-ready brute force and optimal solutions

Use this as an addition to your existing 5-hour master prep PDF. Nothing from the original list is removed.

INTERVIEW FLOW: Clarify -> Restate -> Brute Force -> Limitation -> Optimal Pattern -> Code -> Complexity -> Edge Cases

1. Search in Rotated Sorted Array - LeetCode 33

Question: Given a sorted array of distinct integers that has been rotated at an unknown pivot, and an integer target, return the target index. Return -1 if it does not exist. Your algorithm should run in O(log n) time.

A. Clarification communication

- Are all values distinct? I will assume yes, which lets me determine which half is sorted at every step.
- Can the array be empty? I will return -1 for an empty array.
- Can the array be unrotated? Yes - the same solution should still work.
- Should I return the index rather than the value? I will return the index.
- The required target complexity is O(log n), correct? That suggests a modified binary search.

What I would say: "I want to confirm that values are distinct and I should return the index. Since the array is sorted before rotation and you want O(log n), I am going to look for a binary-search invariant rather than scan linearly."

B. Understanding of the question

A normal sorted array has both halves sorted. After one rotation, at least one side around the middle is still guaranteed to be sorted. That fact tells us which side can safely be discarded.

```
Example:
nums = [4,5,6,7,0,1,2], target = 0

mid = 7 at index 3
left half [4,5,6,7] is sorted
target 0 is NOT inside [4,7]
=> discard left half and search right

Eventually mid reaches value 0 at index 4.
```

C. Brute force approach

English idea: Walk through every element from left to right. As soon as nums[i] equals target, return i. If the loop finishes, return -1.

Data structure used: No additional data structure is required - just an index variable.

Algorithm:

1. Start from index 0.
2. Compare nums[i] with target.
3. If they match, return i.
4. Otherwise continue until the end.
5. Return -1 if no match exists.

Metric	Complexity	Why
Time	O(n)	In the worst case we inspect all n elements.
Space	O(1)	We use only a few scalar variables.

Brute force - fully commented code

```
from typing import List

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        # Check every position one by one.
        for i in range(len(nums)):
            # As soon as we find the target, return its index.
            if nums[i] == target:
                return i

        # Target was not present.
        return -1
```

D. Limitation of brute force

The scan is correct, but it wastes the array's sorted structure. The interviewer explicitly wants O(log n), while the scan is O(n). For 1,000,000 values, linear search may inspect close to one million elements; binary search needs only about 20 iterations.

E. Optimal approach - modified binary search

English idea: At every iteration, compute mid. One of the two halves must be sorted. Determine which half is sorted, check whether the target falls inside that sorted value range, and keep only that half. Otherwise discard it. This removes about half of the remaining search space each time.

Data structures used: No extra collection. Only left, right, and mid indices.

Algorithm:

6. Initialize left = 0 and right = len(nums) - 1.
7. While left <= right, compute mid.
8. If nums[mid] == target, return mid.
9. If nums[left] <= nums[mid], the left half is sorted.
10. If target lies between nums[left] and nums[mid], move right to mid - 1; otherwise move left to mid + 1.
11. Otherwise, the right half is sorted.
12. If target lies between nums[mid] and nums[right], move left to mid + 1; otherwise move right to mid - 1.
13. If the loop ends, return -1.

Optimal - fully commented code

```
from typing import List

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left = 0
        right = len(nums) - 1

        while left <= right:
            # Standard binary-search middle index.
            mid = (left + right) // 2

            # Found the target.
            if nums[mid] == target:
                return mid

            # CASE 1: Left half is sorted.
            # Example: [4, 5, 6, 7, 0, 1, 2]
            #          L       M
            if nums[left] <= nums[mid]:
                # Target lies inside the sorted left half.
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
                else:
                    # Target cannot be in that sorted range.
                    left = mid + 1

            # CASE 2: Right half is sorted.
            else:
                # Target lies inside the sorted right half.
                if nums[mid] < target <= nums[right]:
                    left = mid + 1
                else:
                    # Target cannot be in that sorted range.
                    right = mid - 1

        # Search space became empty.
        return -1
```

Complexity

Metric	Complexity	Why
Time	O(log n)	Every iteration discards about half of the remaining indices.
Space	O(1)	Only left, right, and mid variables are used.

Edge cases to say out loud

- Empty array -> -1.
- Single element: target present or absent.
- Array not rotated at all.
- Target is exactly at the pivot.
- Target is first or last element.
- Target is smaller/larger than all values.

- Two-element arrays.

Pattern trigger: "Rotated sorted array + O(log n)" => modified binary search. Ask: which half is sorted?

2. Lowest Common Ancestor of a Binary Tree - LeetCode 236

Question: Given the root of a binary tree and two nodes p and q, return their lowest common ancestor (LCA). The LCA is the lowest node in the tree that has both p and q as descendants; a node can be a descendant of itself.

A. Clarification communication

- Is this a general binary tree, not a Binary Search Tree? I will not use value ordering.
- Are p and q actual node references that exist in the tree? I will assume both exist unless told otherwise.
- Can one node be an ancestor of the other? Yes - then that ancestor itself is the LCA.
- Are p and q distinct nodes? Usually yes; I will assume they are distinct.
- Do parent pointers exist? I will assume no parent pointers are provided.

What I would say: "Because this is a normal binary tree, values do not tell me which direction to go. I need to reason from tree structure. I will first show a parent-map brute force, then the cleaner postorder DFS solution."

B. Understanding of the question

The answer is the deepest node where the paths to p and q meet. If one subtree contains p and the other contains q, the current node is the LCA. If the current node itself is p or q, that node can also be the answer if the other target is somewhere below it.

```
Example:
      3
     / \
    5   1
   / \
  6   2
     / \
    7   4

p = 5, q = 4

Node 5 is on the path to node 4,
so the lowest common ancestor is 5.
```

C. Brute force approach - parent map + ancestor set

English idea: Traverse the tree once and store each node's parent. Then walk from p upward to the root and put every ancestor into a set. Finally walk from q upward; the first node that already exists in p's ancestor set is the LCA.

Data structures used: A stack or queue to traverse the tree, a hashmap parent[node] = parent_node, and a set containing p's ancestors.

Algorithm:

14. Create parent = {root: None}.
15. Traverse the tree until both p and q have been discovered, storing every child's parent.
16. Walk upward from p and add p, parent[p], parent[parent[p]], ... to a set.
17. Walk upward from q.
18. The first q-side node found in p's ancestor set is the lowest common ancestor.

Metric	Complexity	Why
Time	O(n)	In the worst case we traverse all n nodes; ancestor walks are at most tree height.
Space	O(n)	The parent map can store every node; the stack/set can also grow with the tree.

Brute force - fully commented iterative code

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode',
                             p: 'TreeNode',
                             q: 'TreeNode') -> 'TreeNode':
        # parent[node] tells us how to move upward from a node.
        parent = {root: None}

        # DFS stack used to discover parent relationships.
        stack = [root]

        # Keep traversing until both target nodes are known.
        while p not in parent or q not in parent:
            node = stack.pop()

            if node.left:
                parent[node.left] = node
                stack.append(node.left)

            if node.right:
                parent[node.right] = node
                stack.append(node.right)

        # Store p and all of p's ancestors.
        ancestors = set()
        cur = p
        while cur is not None:
            ancestors.add(cur)
            cur = parent[cur]
```

```
    # Move q upward. The first shared ancestor is the LCA.
    cur = q
    while cur not in ancestors:
        cur = parent[cur]

    return cur
```

D. Limitation of brute force

The parent-map approach is easy to reason about and is iterative, but it stores a parent entry for potentially every node plus an ancestor set. We can solve the problem more directly using the recursive structure of the tree and avoid the O(n) parent map.

E. Optimal approach - postorder DFS

English idea: Ask each subtree whether it found p or q. If the current node is None, return None. If the current node is p or q, return it. Recursively search left and right. If both sides return non-None nodes, p and q split across the two sides, so the current node is the LCA. If only one side returns non-None, pass that result upward.

Data structure used: No explicit collection is required. The recursive call stack stores the path through the tree.

Algorithm:

19. Base case: if root is None, return None.
20. If root is p or q, return root immediately.
21. Recursively search the left subtree.
22. Recursively search the right subtree.
23. If both left and right return a node, the current root is where the two targets meet, so return root.
24. Otherwise return whichever side is non-None.

Optimal - fully commented code

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode',
                             p: 'TreeNode',
                             q: 'TreeNode') -> 'TreeNode':
        # Nothing to find below an empty node.
        if root is None:
            return None

        # Important: a node can be an ancestor of itself.
        # If the current node is p or q, return it upward.
        if root == p or root == q:
            return root
```

```
    # Search both subtrees.
    left = self.lowestCommonAncestor(root.left, p, q)
    right = self.lowestCommonAncestor(root.right, p, q)

    # If we found one target on each side, this root is
    # the first/lowest point where their paths meet.
    if left is not None and right is not None:
        return root

    # Otherwise both targets are in the same subtree.
    # Return the non-None result upward.
    if left is not None:
        return left

    return right
```

Complexity

Metric	Complexity	Why
Time	O(n)	Each node is visited at most once in the worst case.
Space	O(h)	The recursion depth is the tree height h. Balanced tree: O(log n); skewed tree: O(n).

Why this code correctly handles p being an ancestor of q

Suppose p = node 5 and q = node 4. When DFS reaches node 5, it immediately returns node 5 because root == p. That result travels upward. Since q is inside node 5's subtree, node 5 is correctly preserved as the answer. This is exactly why the base case "root == p or root == q" is important.

Edge cases to say out loud

- p is the root.
- q is the root.
- p is an ancestor of q.
- q is an ancestor of p.
- p and q are in different root subtrees.
- Highly skewed tree - recursion depth becomes O(n).
- If the interviewer removes the guarantee that both nodes exist, this implementation needs extra existence tracking.

Pattern trigger: "General binary tree + relationship between two nodes" => think postorder DFS. If left finds one and right finds one, current node is the meeting point.

2-Minute Final Revision

Question	Trigger	Core sentence to remember
LC 33 - Rotated Array	O(log n), rotated + distinct	One half is always sorted. Check whether target belongs to that sorted half, then discard the other half.

Question	Trigger	Core sentence to remember
LC 236 - LCA	General binary tree, two nodes	Postorder: if left and right both find something, current root is LCA; otherwise return the non-None side.

Interview reminder: Do not jump directly into code. Clarify assumptions, state the brute force, explain why it is insufficient, name the optimal pattern, then code while narrating the invariant.